

Introduction to Algorithms

Levon Prazyan

Course Overview

- **Introduction**
 - 1 lecture
- **Analyzing Algorithms**
 - 2 lectures
- **Recurrences**
 - 2 lectures
- **Probabilistic Analysis and Randomized Algorithms**
 - 2 lecture
- **Sorting and Searching**
 - 4 lectures
- **Heaps**
 - 1 lecture
- **Arrays, Lists**
 - 1 lectures
- **Stack, Queue, Double Ended Queue**
 - 2 lectures

Algorithm. Key Characteristics

An **algorithm** is a well-defined, step-by-step procedure or set of rules for solving a specific problem or performing a task.

- **Finiteness.**
- **Definiteness.**
- **Input.**
- **Output.**
- **Effectiveness.**
- **Generality.**

Correctness

An algorithm is said to be ***correct*** if, for every input instance, it halts with the correct output.

An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer.

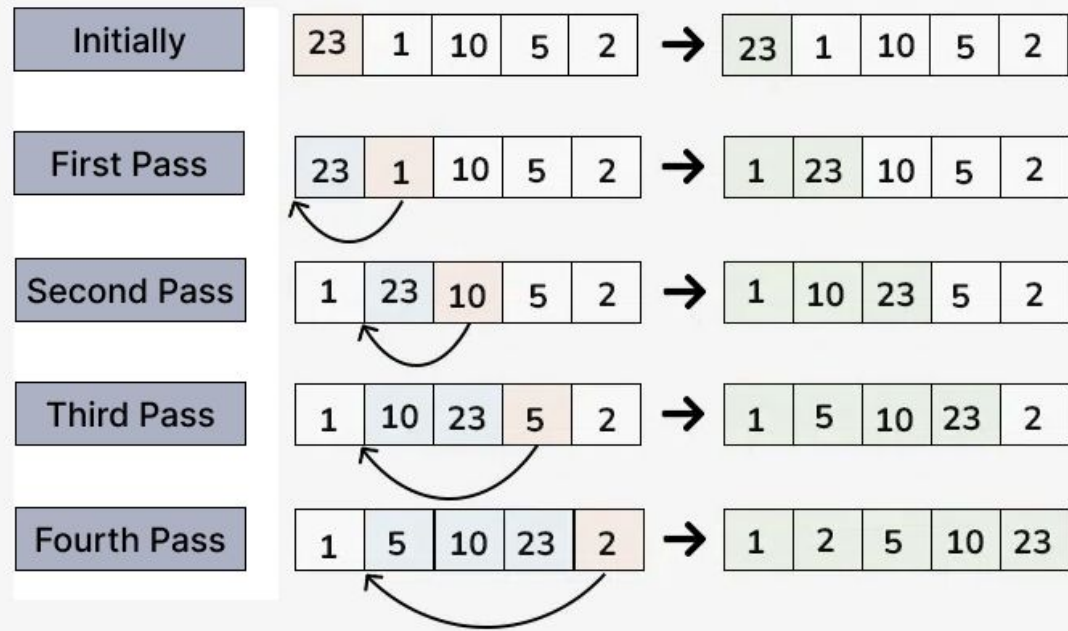
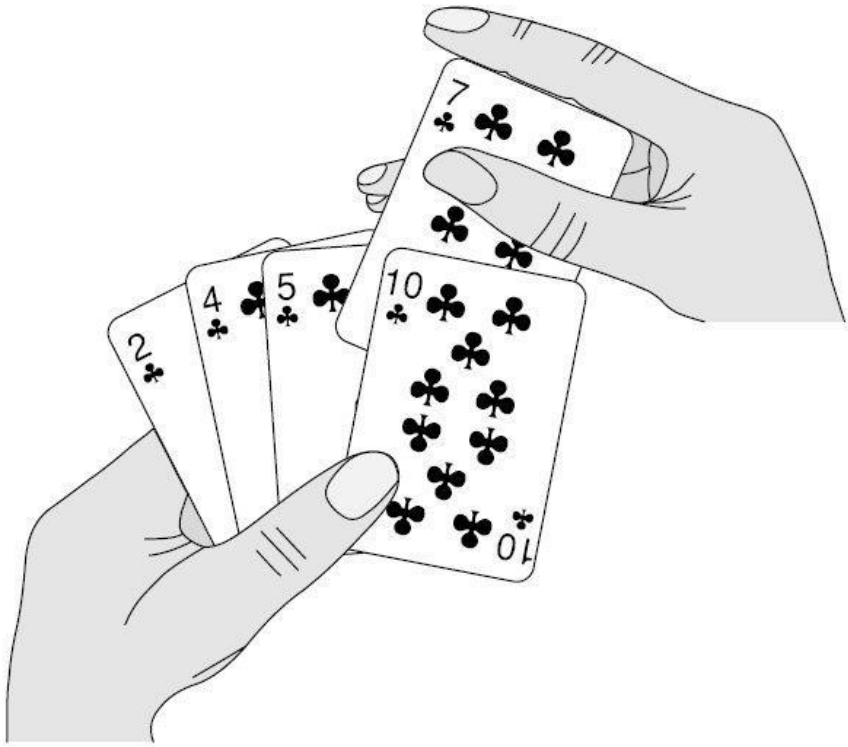
Sorting Problem

-

- **Input:** A sequence of n keys $(a_1, a_2, a_3, \dots, a_n)$
- **Output:** A reordering $(a'_1, a'_2, a'_3, \dots, a'_n)$ such that

$$a'_1 \leq a'_2 \leq a'_3 \leq \dots \leq a'_n$$

Insertion Sort



Insertion Sort

6 5 3 1 8 7 2 4

Insertion Sort: Pseudocode

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$
2. $key = A[j]$
3. // Insert $A[j]$ into the sorted sequence $A[1 .. j-1]$.
4. $i = j - 1$
5. **while** $i > 0$ and $A[i] > key$
6. $A[i+1] = A[i]$
7. $i = i - 1$
8. $A[i+1] = key$

Loop invariant:

*$A[1 .. j-1]$ are the elements
originally in positions 1 through $j-1$,
but now in sorted order.*

Insertion Sort: Correctness: Proof

Initialization: The loop invariant is true prior to the first iteration of the loop.

Maintenance: If it is true before an iteration of the loop, it remains true before the next iteration.

Termination: When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

Analyzing Algorithms

- **Analyzing** an algorithm has come to mean predicting the resources that the algorithm requires.

1. **time**
2. **memory**
3. communication bandwidth
4. hardware
5. ...

A model of the implementation technology that we will use is a generic one-processor, random-access machine (RAM) model of computation.

Insertion Sort: Analyzing (1)

The **running time** of a program is a function of the **size** of its input.

- The best notion for input size depends on the problem being studied.
- The running time of an algorithm on a particular input is the number of primitive operations or “steps” executed.
 - A constant amount of time is required to execute each line of the pseudocode.
 - assume that each execution of the i^{th} line takes time c_i , where c_i is a constant.

Insertion Sort: Analyzing (2)

	InsertionSort	Cost	Times
1	for $j = 2$ to $A.length$		
2	$key = A[j]$		
3	// Insert $A[j]$ into the sorted sequence $A[1 .. j-1]$.		
4	$i = j - 1$		
5	while $i > 0$ and $A[i] > key$		
6	$A[i+1] = A[i]$		
7	$i = i - 1$		
8	$A[i+1] = key$		

Insertion Sort: Analyzing (3)

$$\bullet T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

Even for inputs of a given size, an algorithm's running time may depend on *which* input of that size is given.

- When the initial array is sorted
 $T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1); T(n) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$

- When the initial array is reverse sorted: $t_j = j; \sum_{j=2}^n j = \left(\frac{n(n+1)}{2}\right) - 1; \sum_{j=2}^n (j-1) = \left(\frac{n(n-1)}{2}\right)$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1\right) + c_6 \left(\frac{n(n-1)}{2}\right) + c_7 \left(\frac{n(n-1)}{2}\right) + c_8(n-1)$$

$$T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right) n - (c_2 + c_4 + c_5 + c_8)$$

$$T(n) = an^2 + bn + c$$

Insertion Sort: Analyzing (3)

- $$T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1)$$

Insertion Sort: Analyzing (4)

Even for inputs of a given size, an algorithm's running time may depend on *which* input of that size is given.

- When the initial array is sorted

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$$

$$T(n) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

- When the initial array is reverse sorted:

$$t_j = j; \sum_{j=2}^n j = \left(\frac{n(n+1)}{2}\right) - 1; \sum_{j=2}^n (j-1) = \left(\frac{n(n-1)}{2}\right)$$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1\right) + c_6 \left(\frac{n(n-1)}{2}\right) + c_7 \left(\frac{n(n-1)}{2}\right) + c_8(n-1)$$

$$T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right) n - (c_2 + c_4 + c_5 + c_8)$$

$$T(n) = \mathbf{an^2 + bn + c}$$

Worst-case and Average-case Analysis

Worst-case running time is the longest running time for any input of size n .

- Knowing it provides a guarantee that the algorithm will never take any longer.
- For some algorithms, the worst case occurs often.
- The “average case” is often roughly as bad as the worst case.

The scope of average-case analysis is limited, because it may not be apparent what constitutes an “average” input for a particular problem.

Order of Growth

- $T(n) = an^2 + bn + c$
- The **rate of growth** or **order of growth**, of the running time is the leading term of a formula.
 - Ignore lower-order terms: $an^2 + bn + c$
 - ignore leading term's constant coefficient: (an^2)

The insertion sort has a worst-case running time of $\Theta(n^2)$

One algorithm to be **more efficient** than another if its worst-case running time has a lower order of growth.

Recursion

- **Recursion** is a method of defining a function, procedure, or algorithm in which the entity is specified in terms of itself, with at least one base case (non-recursive termination condition) and one or more recursive cases (self-referential calls).

Formally, a recursive function f over a domain D satisfies:

1. Base case: $f(d)$ is defined directly for one or more specific values $d \in D$.
2. Recursive case: For other values of $d \in D$, $f(d)$ is defined in terms of one or more evaluations of $f(d')$, where d' is “smaller” or “simpler” than d .

The Divide-and-conquer Approach

Many useful algorithms are **recursive** in structure. These algorithms typically follow a **divide-and-conquer** approach:

- **Divide** the problem into several subproblems that are similar to the original problem but smaller in size
- **Conquer** the subproblems recursively.*
- **Combine** solutions to create a solution to the original problem.

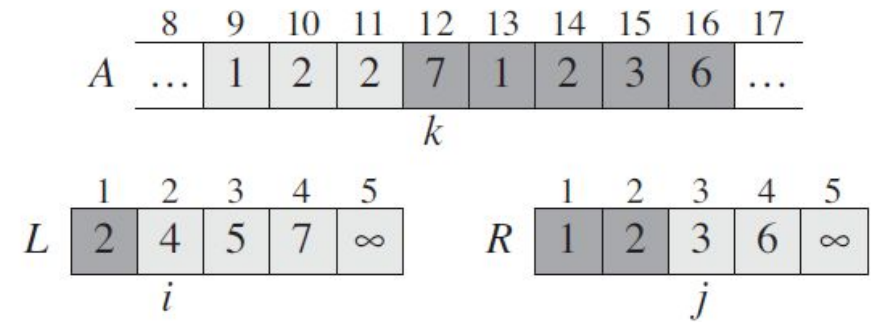
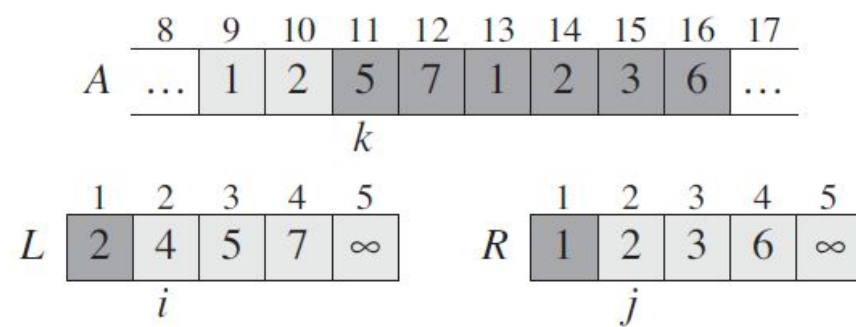
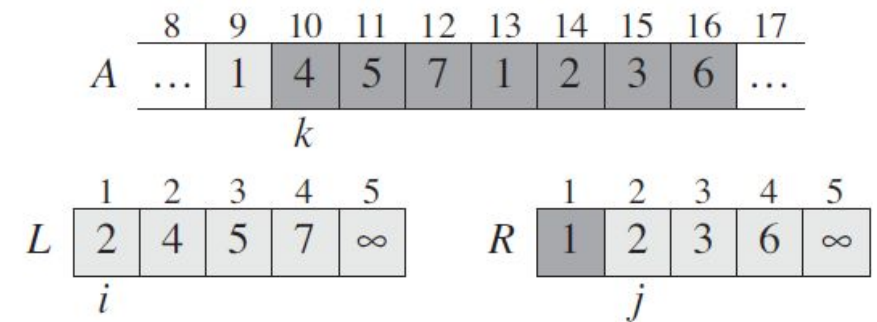
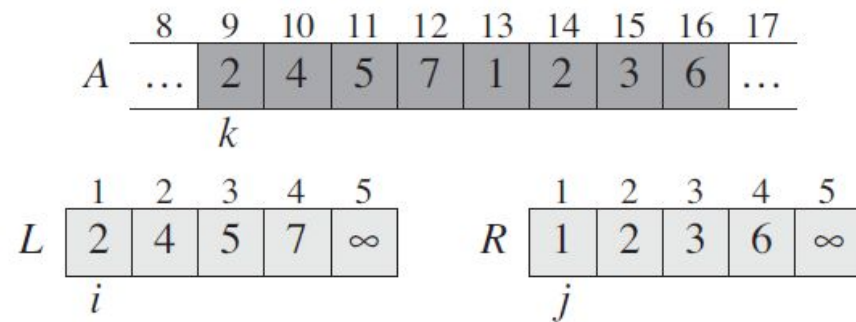
Merge Sort

The merge sort algorithm closely follows the divide-and-conquer paradigm.

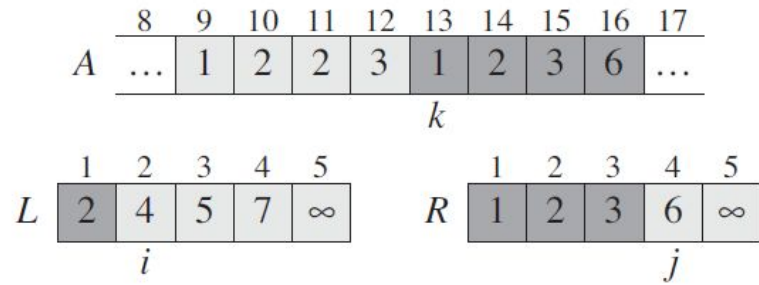
- Divide:** Divide the n -element sequence to be sorted into two subsequences of $n/2$ elements each.
- Conquer:** Sort the two subsequences recursively using merge sort.
- Combine:** Merge the two sorted subsequences to produce the sorted answer.

The recursion “bottoms out” when the sequence to be sorted has length 1

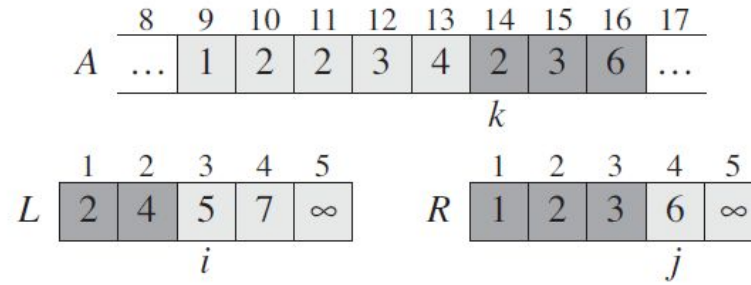
Merge Sort: Merge(A, p, q, r)



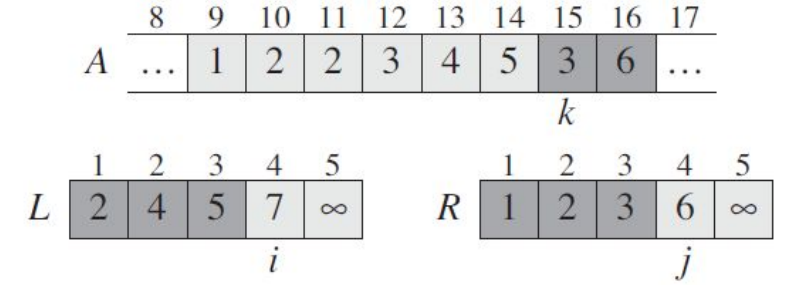
Merge Sort: Merge(A, p, q, r)



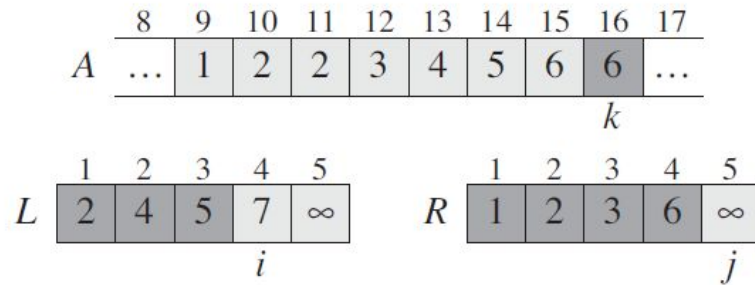
(e)



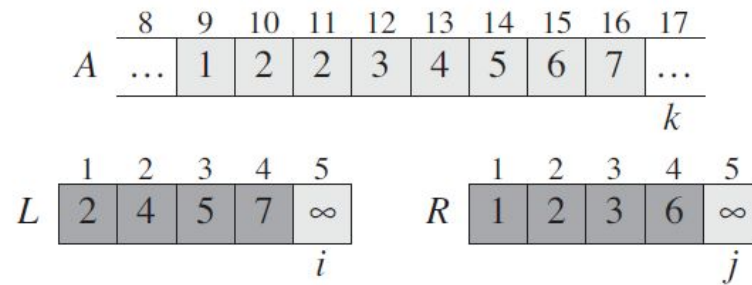
(f)



(g)



(h)



(i)

Merge Sort: Merge

10	50	75	80
----	----	----	----

55	67	79	90
----	----	----	----

Merge Sort: Merge(A, p, q, r)

MERGE(A, p, q, r)

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$ 
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

- $A[p..q]$ and $A[q+1 .. r]$ are sorted.

- Merge takes $\Theta(n)$ time. $n = r-p+1$

Invariant: At the start of each iteration of the for loop of lines 12–17, the subarray $A[p .. k - 1]$ contains the $k - p$ smallest elements of L and R , in sorted order. $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A .

- **Initialization:**

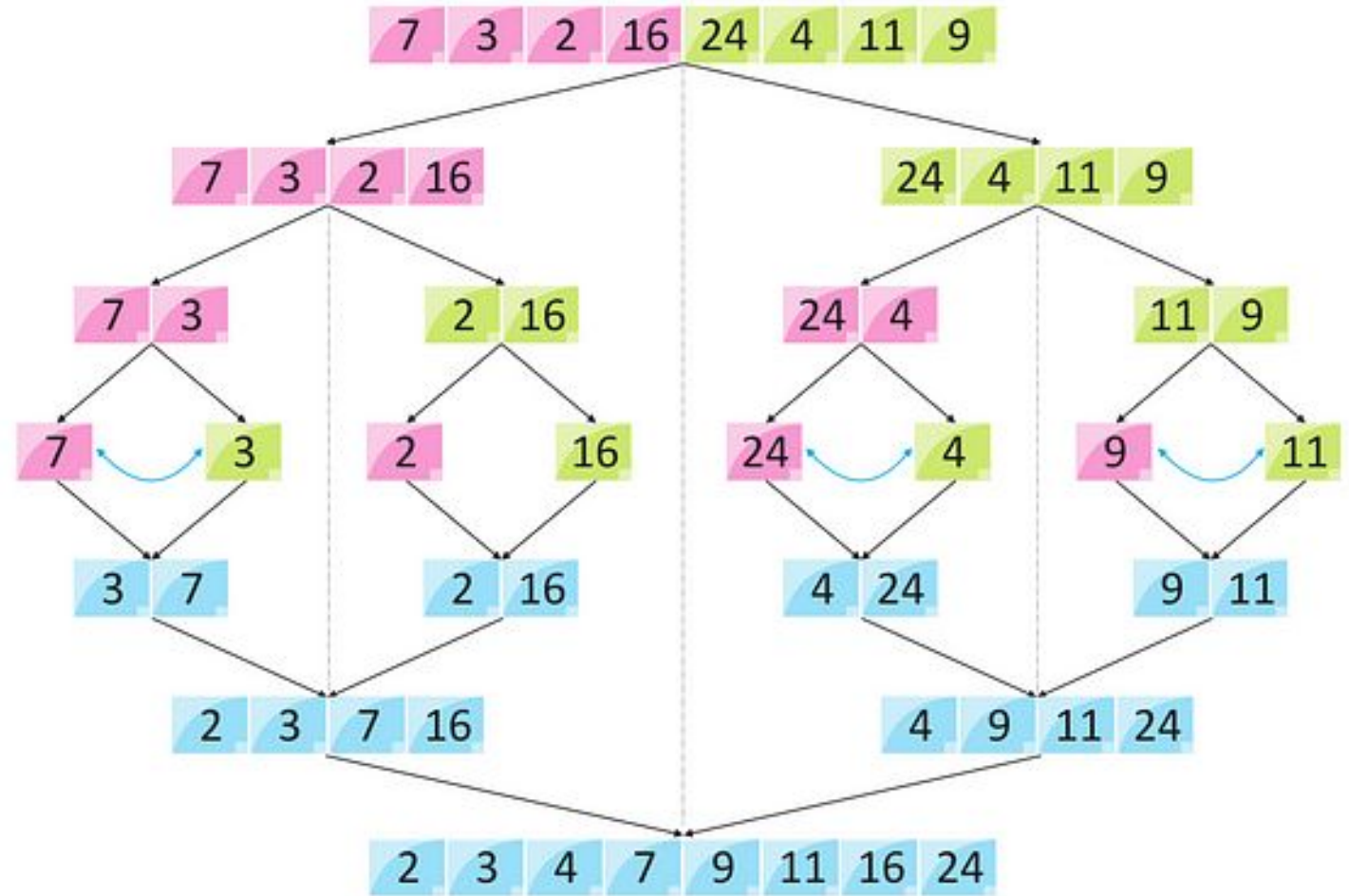
- **Maintenance:**

- **Termination:**

Merge Sort

MERGE-SORT(A, p, r)

- 1 **if** $p < r$
- 2 $q = \lfloor (p + r) / 2 \rfloor$
- 3 MERGE-SORT(A, p, q)
- 4 MERGE-SORT($A, q + 1, r$)
- 5 MERGE(A, p, q, r)



Merge Sort

6 5 3 1 8 7 2 4

Analyzing Merge Sort (1)

- **Divide:** The divide step just computes the middle of the subarray, which takes constant time.

$$D(n) = \Theta(1)$$

- **Conquer:** We recursively solve two subproblems, each of size $n/2$, which contributes $2T(n/2)$ to the running time.
- **Combine:** The MERGE procedure on an n -element subarray takes time $\theta(n)$.

$$C(n) = \Theta(n)$$

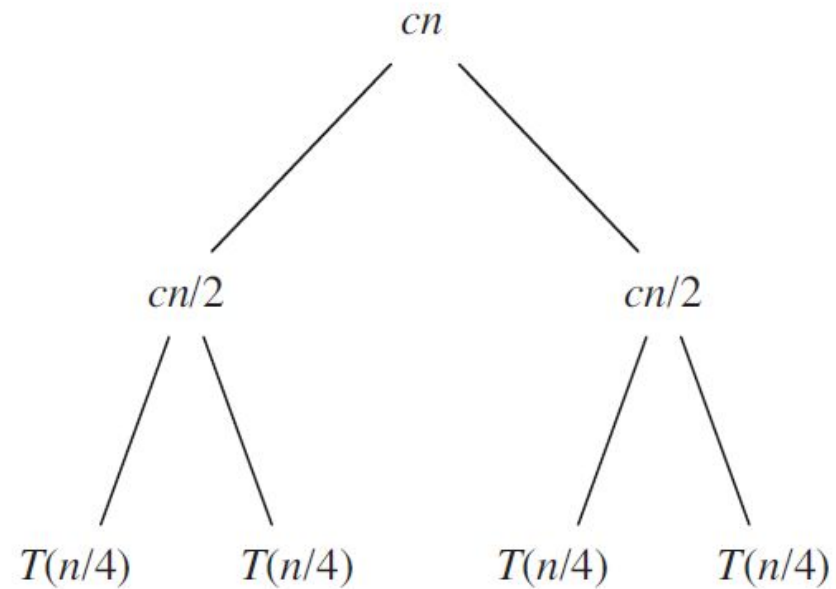
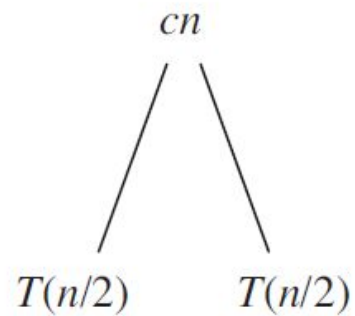
$$T(n) = \begin{cases} \Theta(1); & n = 1 \\ 2T(n/2) + \Theta(n); & n > 1 \end{cases}$$

$$T(n) = \Theta(n \cdot \log n)$$

Analyzing Merge Sort (2)

- $$T(n) = \begin{cases} cn; & n = 1 \\ 2T(n/2) + cn; & n > 1 \end{cases}$$

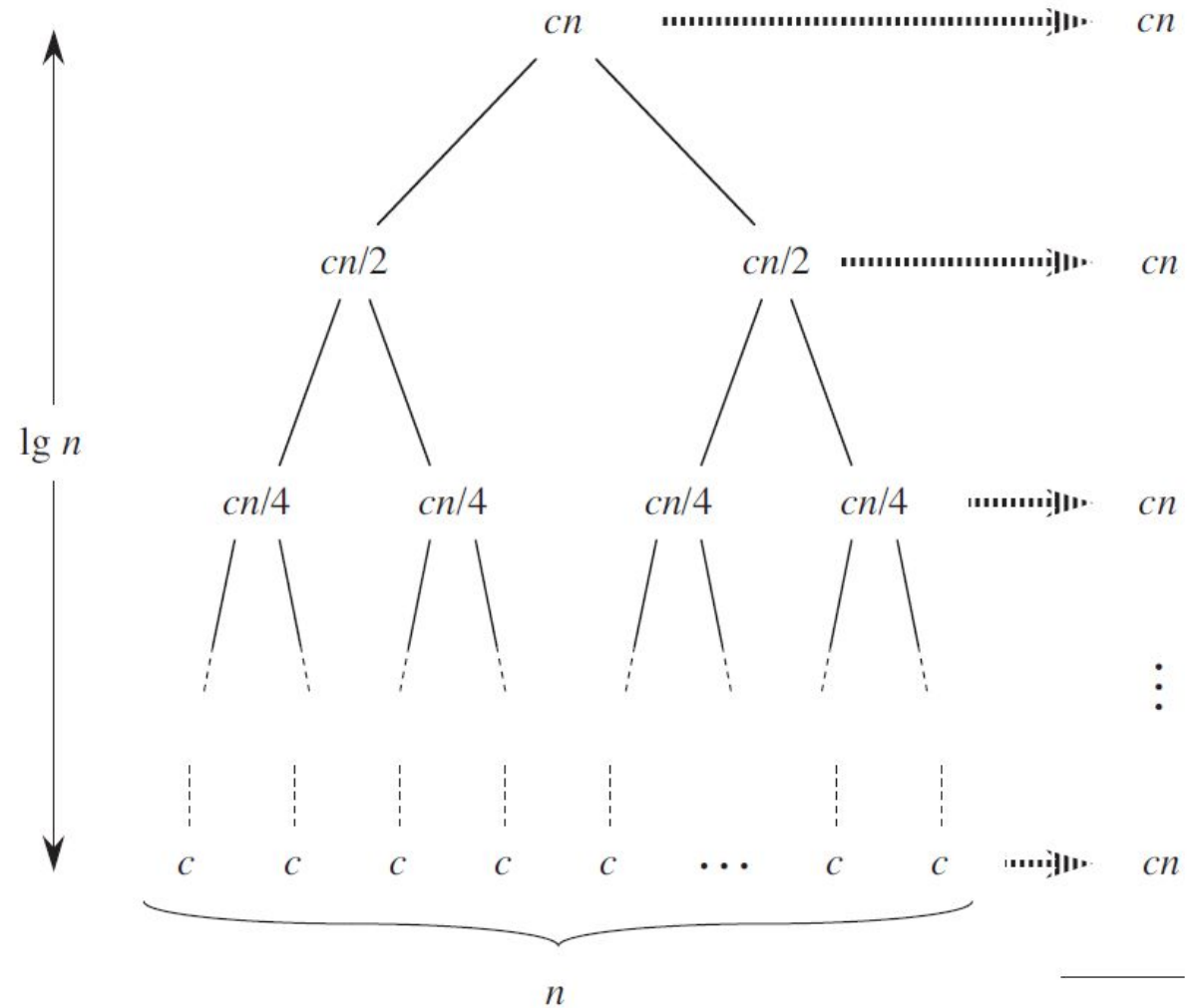
$T(n)$



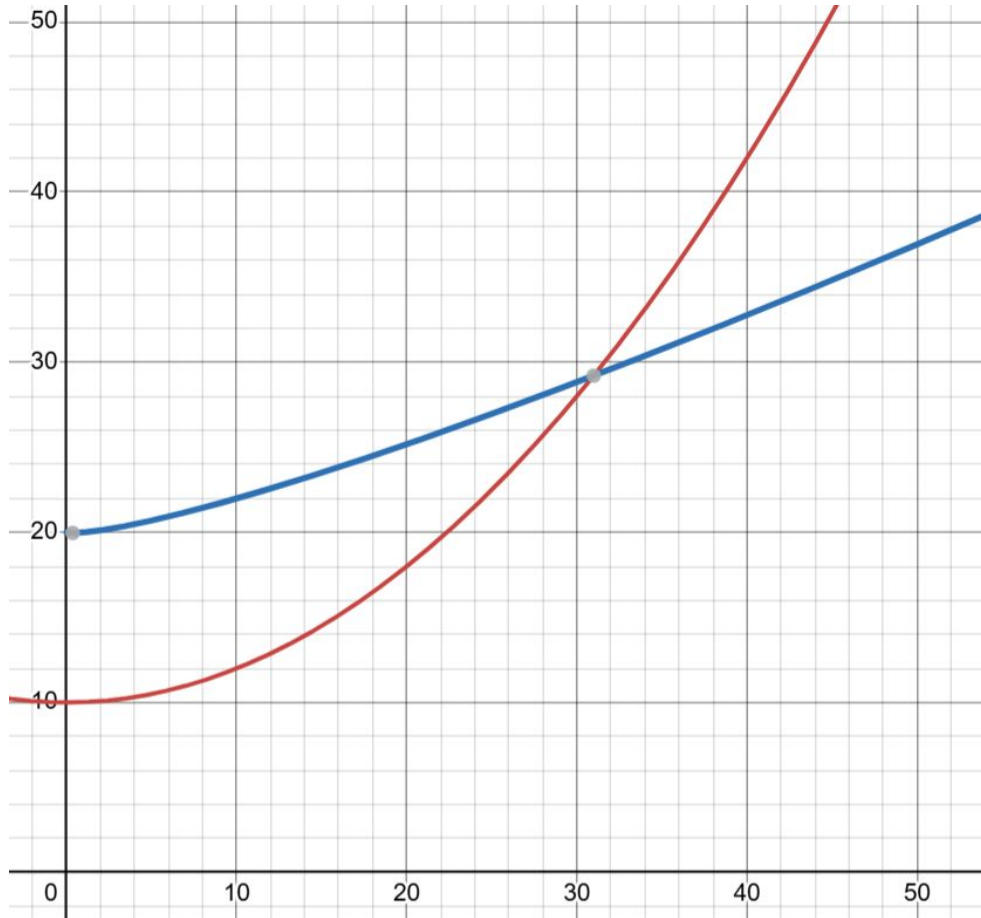
Analyzing Merge Sort (3)

- $T(n) = cn \log n + cn$

$$T(n) = \Theta(n \cdot \log n)$$



The Growth of The Running Time



Worst-case:

- Merge Sort - $\Theta(n \cdot \log n)$
- Insertion Sort - $\Theta(n^2)$

Asymptotic Efficiency

The **growth of the running time of an algorithm** refers to how the execution time of an algorithm increases as the size of its input grows.

- Growth order

The **asymptotic efficiency** of an algorithm refers to its performance in terms of time complexity and space complexity as the input size (n) becomes very large.

- Asymptotic efficiency

Θ Notation (1)

For a given function $g(n)$

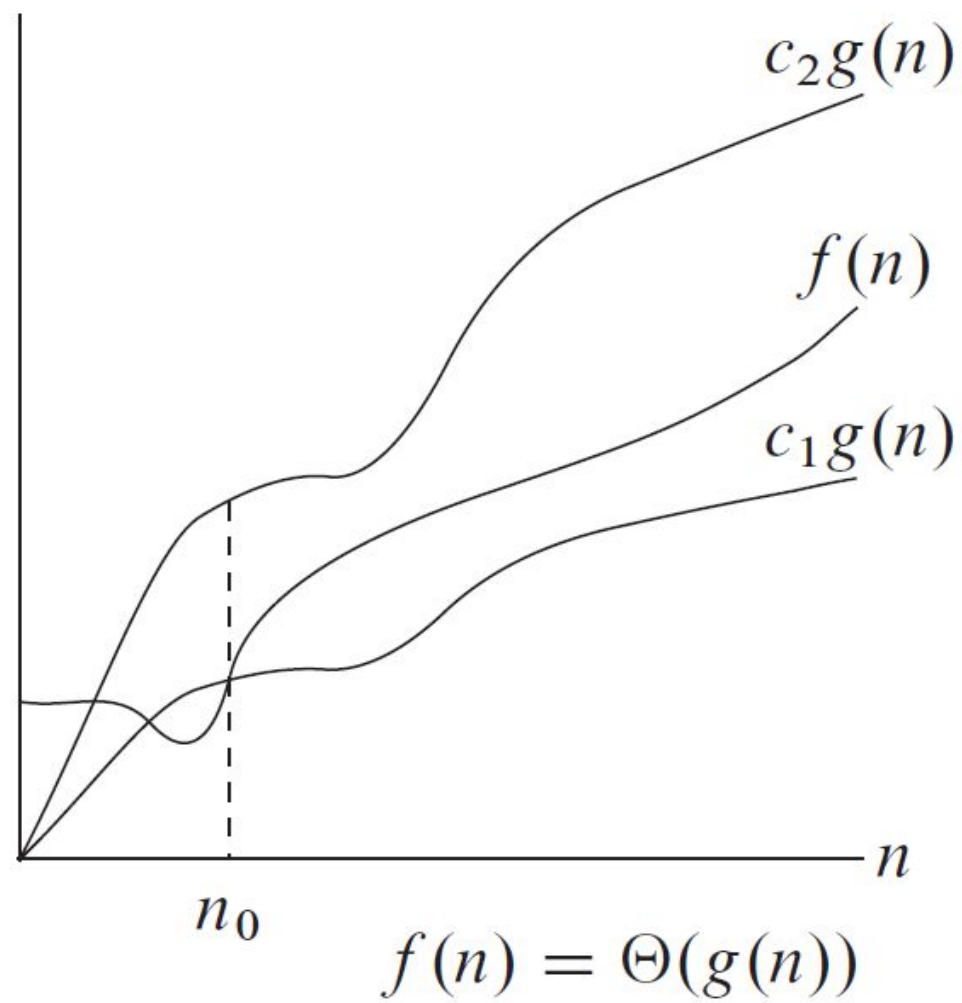
$$\Theta(g(n)) = \{f(n): \exists c_1, c_2, n_0: 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), n \geq n_0\}$$
$$c_1 > 0, c_2 > 0, n_0 > 0$$

$\Theta(g(n))$ is a set of functions.

By writing $f(n) = \Theta(g(n))$ we mean $f(n) \in \Theta(g(n))$

$g(n)$ is an **asymptotic tight bound** for $f(n)$.

Θ Notation (2)



Θ Notation: Example 1

Let: $f(n) = 3n^2 - 2n$

Show: $f(n) = \Theta(n^2)$

$$c_1 n^2 \leq 3n^2 - 2n \leq c_2 n^2; c_1 > 0, c_2 > 0, n_0 > 0$$

$$c_1 \leq 3 - \frac{2}{n} \leq c_2$$

$$n_0 = 1; c_1 = \frac{1}{2}; c_2 = 4$$

$$\frac{1}{2}n^2 \leq 3n^2 - 2n \leq 4n^2; n \geq 1$$

O notation (1)

For a given function $g(n)$

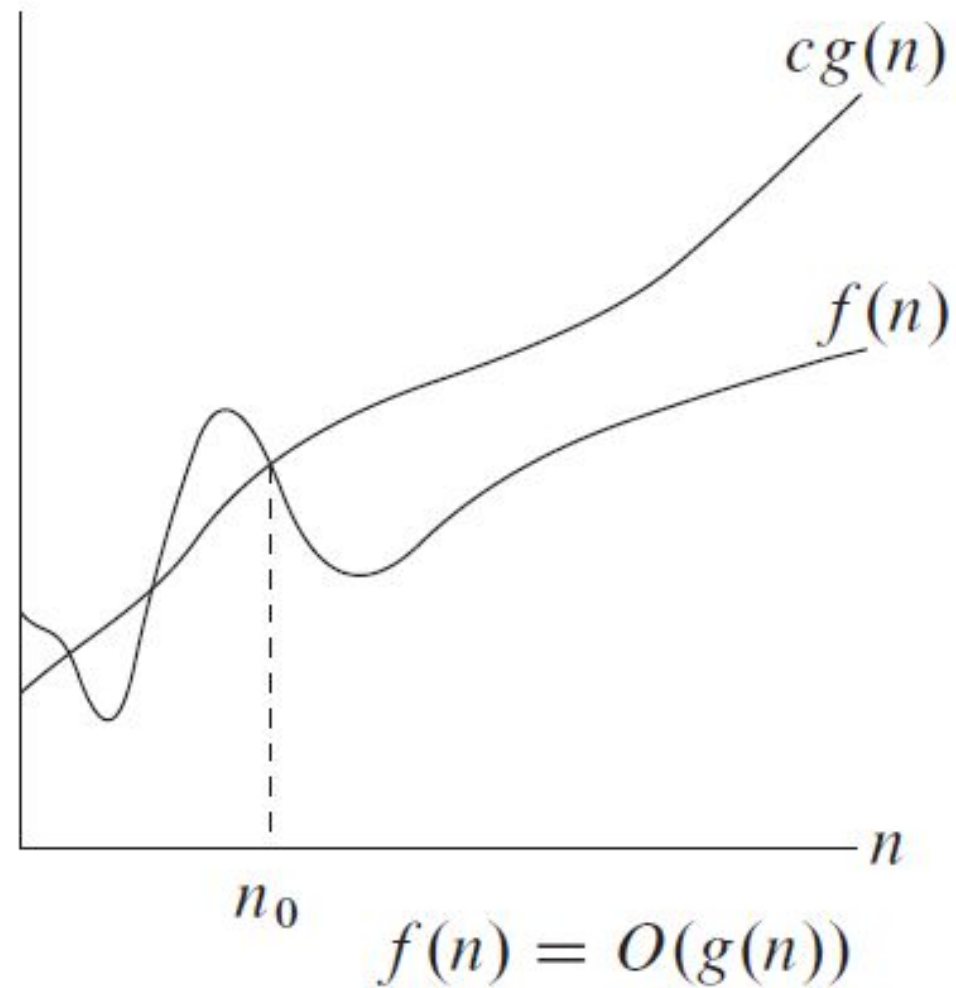
$$O(g(n)) = \{f(n) : \exists c, n_0 : 0 \leq f(n) \leq cg(n), n \geq n_0\}$$
$$c > 0, \quad n_0 > 0$$

$O(g(n))$ is a set of functions.

By writing $f(n) = O(g(n))$ we mean $f(n) \in O(g(n))$

$g(n)$ is an **asymptotic upper bound** for $f(n)$.

O Notation (2)



Ω notation (1)

For a given function $g(n)$

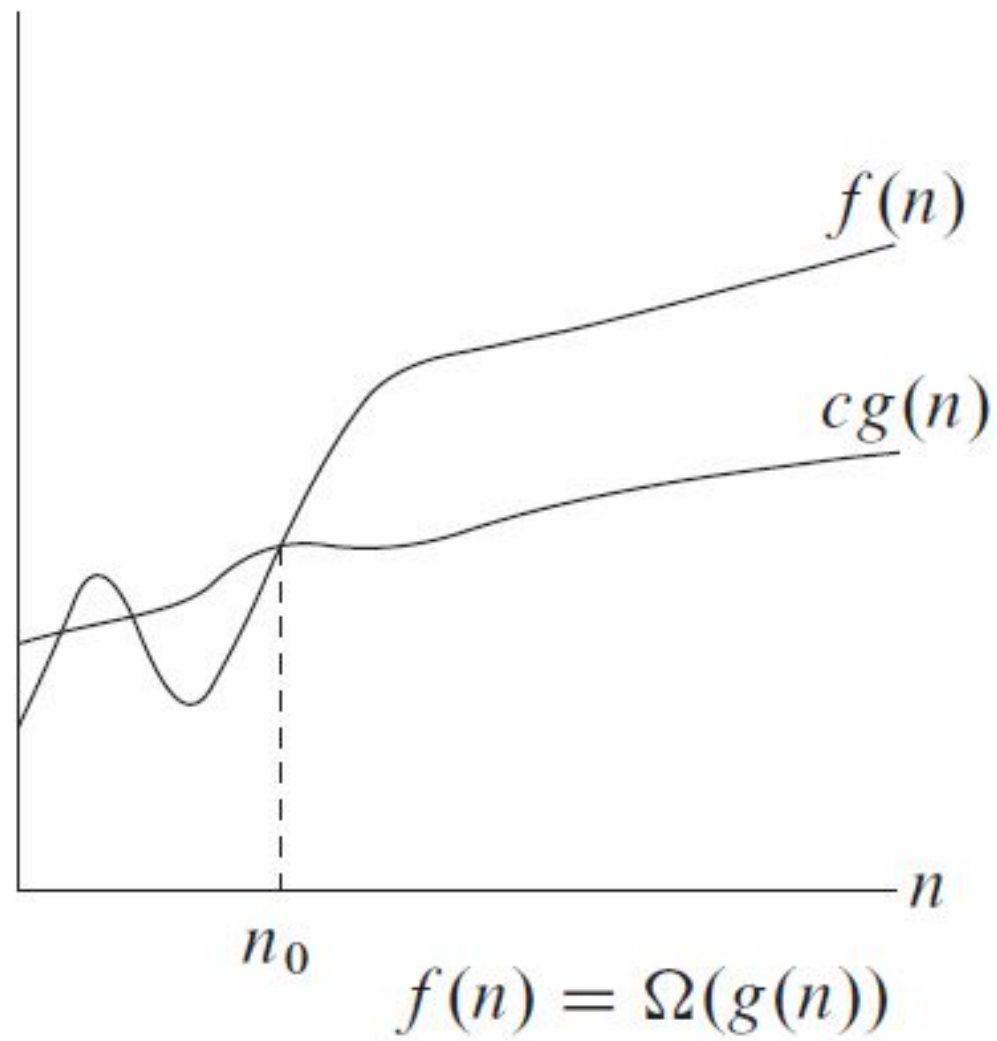
$$\Omega(g(n)) = \{f(n) : \exists c, n_0 : 0 \leq cg(n) \leq f(n), n \geq n_0\}$$
$$c > 0, \quad n_0 > 0$$

$\Omega(g(n))$ is a set of functions.

By writing $f(n) = \Omega(g(n))$ we mean $f(n) \in \Omega(g(n))$

$g(n)$ is an **asymptotic lower bound** for $f(n)$.

Ω Notation (2)



Theorem

• For any two functions $f(n)$ and $g(n)$, we have

$$f(n) = \Theta(g(n))$$

if and only if

$$f(n) = O(g(n))$$

and

$$f(n) = \Omega(g(n))$$

o notation

For a given function $g(n)$

$$o(g(n)) = \{f(n) : \forall c, \exists n_0 : 0 \leq f(n) \leq cg(n), n \geq n_0\}$$
$$c > 0, \quad n_0 > 0$$

$o(g(n))$ is a set of functions.

By writing $f(n) = o(g(n))$ we mean $f(n) \in o(g(n))$

$g(n)$ is an **asymptotic upper NOT tight bound** for $f(n)$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

$$3n + 2 = o(n^2); \quad 3n + 2 \neq o(n)$$

ω notation

For a given function $g(n)$

$$\omega(g(n)) = \{f(n) : \forall c, \exists n_0 : 0 \leq cg(n) \leq f(n), n \geq n_0\}$$
$$c > 0, \quad n_0 > 0$$

$\omega(g(n))$ is a set of functions.

By writing $f(n) = \omega(g(n))$ we mean $f(n) \in \omega(g(n))$

$g(n)$ is an **asymptotic lower NOT tight bound** for $f(n)$.

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$$

$$4n^3 + 6n^2 + 7 = \omega(n^2) ; 4n^3 + 6n^2 + 7 \neq \omega(n^3)$$

Asymptotic Notations

For a given function $g(n)$

- $\Theta(g(n)) = \{f(n) : \exists c_1, c_2, n_0 : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), n \geq n_0\}$
- $O(g(n)) = \{f(n) : \exists c, n_0 : 0 \leq f(n) \leq c g(n), n \geq n_0\}$
- $\Omega(g(n)) = \{f(n) : \exists c, n_0 : 0 \leq c g(n) \leq f(n), n \geq n_0\}$
- $o(g(n)) = \{f(n) : \forall c, n_0 : 0 \leq f(n) \leq c g(n), n \geq n_0\}$
- $\omega(g(n)) = \{f(n) : \forall c, n_0 : 0 \leq c g(n) \leq f(n), n \geq n_0\}$

$$c_1, c_2, c, n_0 > 0$$

Asymptotic Notation in Equations

- $$3n^2 + \theta(n) = \theta(n^2)$$

No matter how the anonymous functions are chosen on the left of the equal sign, there is a way to choose the anonymous functions on the right of the equal sign to make the equation valid.

For every $\forall f(n) = \theta(n)$ there is such $\exists g(n) = \theta(n^2)$ that

$$3n^2 + f(n) = g(n)$$

Transitivity of Asymptotic Notations

- $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$
 $f(n) = O(g(n))$ and $g(n) = O(h(n)) \implies f(n) = O(h(n))$
 $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n)) \implies f(n) = \Omega(h(n))$
 $f(n) = o(g(n))$ and $g(n) = o(h(n)) \implies f(n) = o(h(n))$
 $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n)) \implies f(n) = \omega(h(n))$

Reflexivity of Asymptotic Notations

-

$$f(n) = \Theta(f(n))$$

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

Symmetry of Asymptotic Notations

$\dot{f}(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$

$f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$

$f(n) = \Omega(g(n))$ if and only if $g(n) = O(f(n))$

Exercises

1. Let $f(n)$ and $g(n)$ be asymptotically nonnegative functions.
Prove that $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.
2. Show that for any real constants a and b , where $b > 0$
$$(n + a)^b = \Theta(n^b)$$
3. Is $2^{n+2} + 2 = O(2^n)$? Is $2^{2n+2} + 2 = O(2^n)$?
4. Prove that $o(g(n)) \cap \omega(g(n)) = \emptyset$.



Thank You